LLM and AI-agent foundations

Foundations 1: University teaching deck

How does a language model become the decision-making core of an agent?

Agents, tokens, Transformers, training, reasoning, and the limits of model capability.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Agent foundations

Identify the environment, observations, actions, and decision process in an LLM-backed agent. Explain how autonomy differs across systems.

Model foundations

Trace a prompt through tokenization, embeddings, Transformer blocks, and token selection. Connect training stages to instruction following and reasoning.

Reasoning foundations

Compare prompting, multiple-candidate search, and trained reasoning. Explain why more computation, a larger context window, or a longer answer does not guarantee success.

Source: Ch. 1–3, PDF pp. 15–185

Agents connect observations to actions

Environment

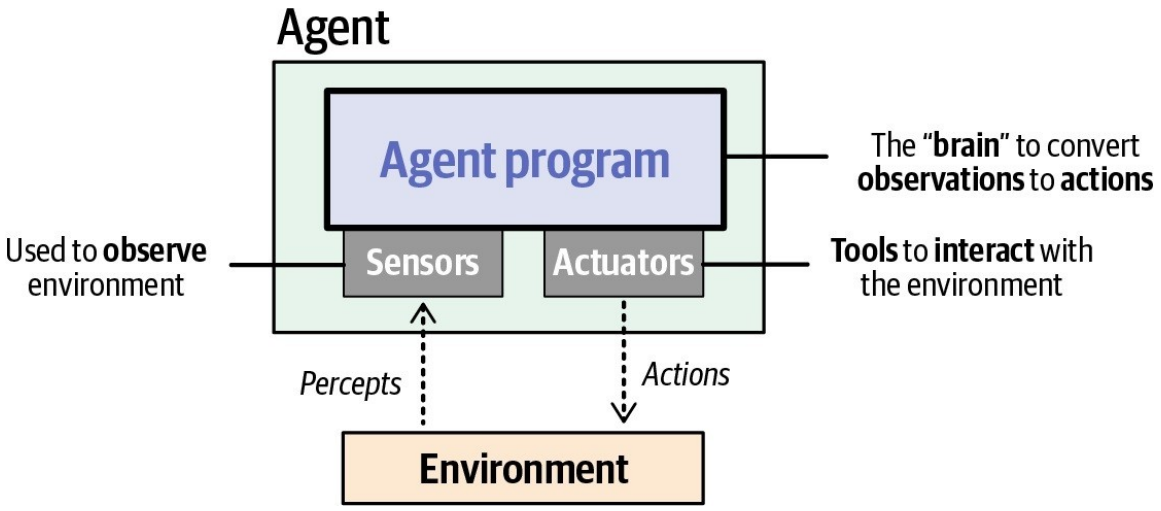
The environment is the world a system can observe and affect. It may be a website, a repository, a database, or a physical setting.

Perception and action

Sensors provide observations. Actuators carry out actions. An agent program decides what action to take from the available observations.

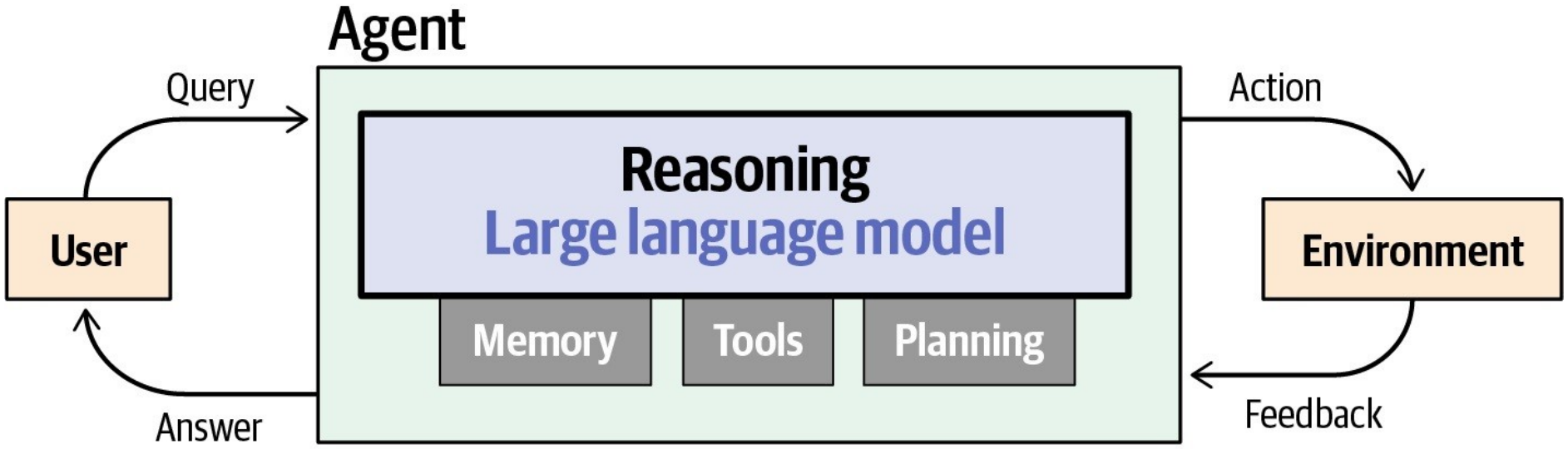
LLM-backed agents

A language model can supply the decision process. Its tools connect textual decisions to changes or information in the environment.



Grootendorst & Alammar (2026). Fig. 1-1, PDF p. 16

The anatomy of an LLM-backed agent



Reasoning core
The LLM interprets a goal and chooses what to do next. It operates inside software that supplies context and executes actions.

Augmentations
Memory retains useful information. Tools retrieve information or take action. Planning decomposes a goal, and reflection revises an approach after feedback.

System boundary
The user, the agent, and the environment interact over repeated steps. The quality of the whole system depends on how these components work together.

Grootendorst & Alammar (2026). Fig. 1-2, PDF p. 17

Autonomy exists on a spectrum

Limited independence

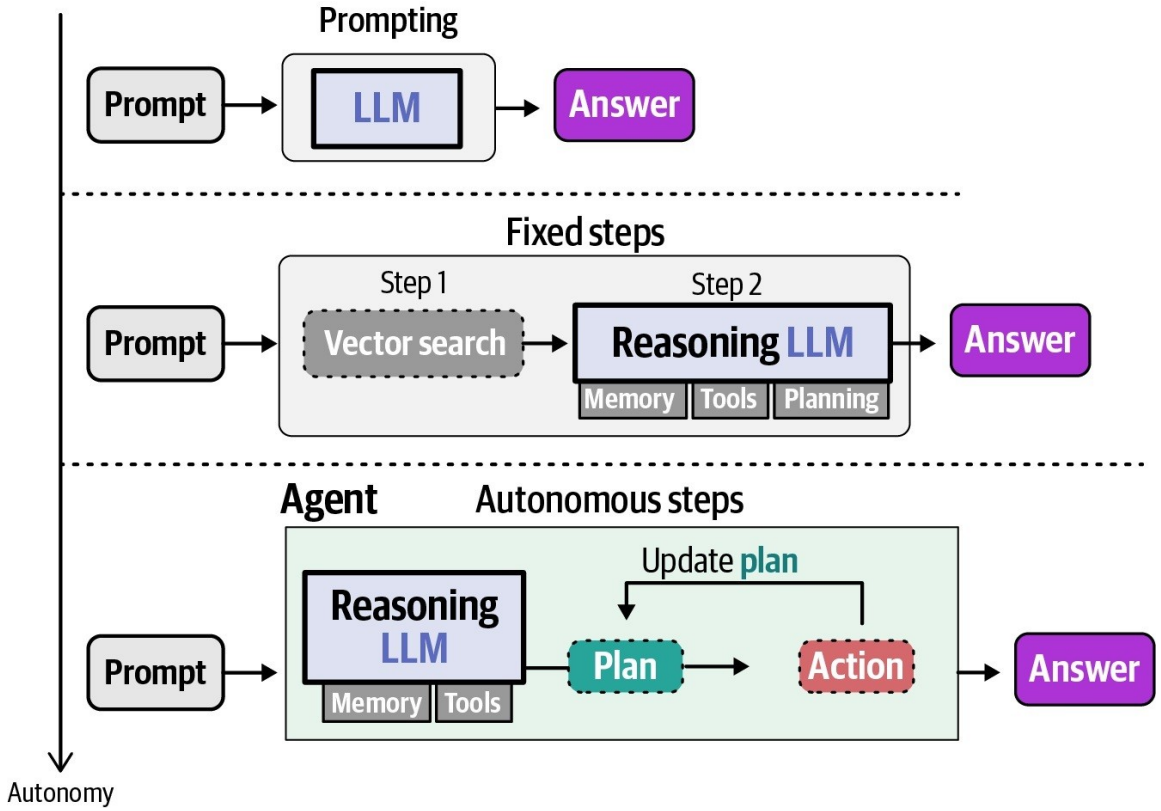
A system may generate one answer or choose one tool while an application controls the surrounding workflow.

Greater independence

An agent may decide which tools to use, how many steps to take, when to revise its plan, and when to stop.

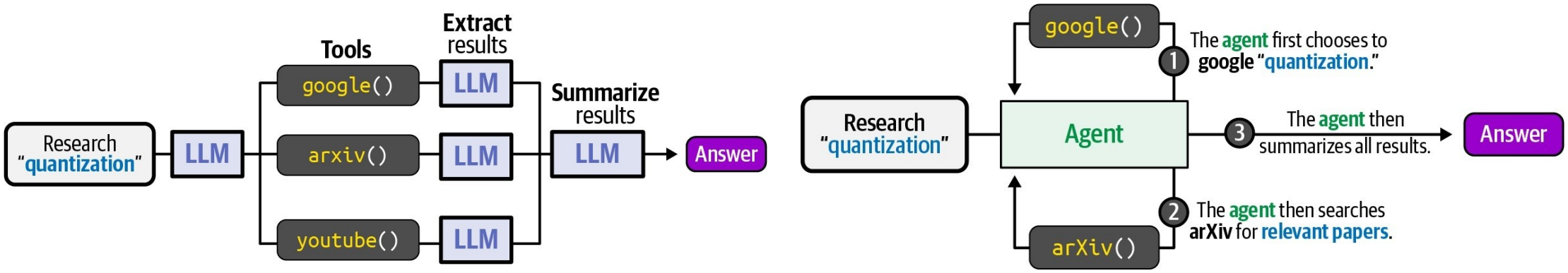
Design implication

Goal-directed decisions can appear inside an orchestrated workflow. The appropriate degree of autonomy depends on the task and the consequences of a mistaken action.



Grootendorst & Alammar (2026). Fig. 1-19, PDF p. 34

Fixed workflows and adaptive tool use



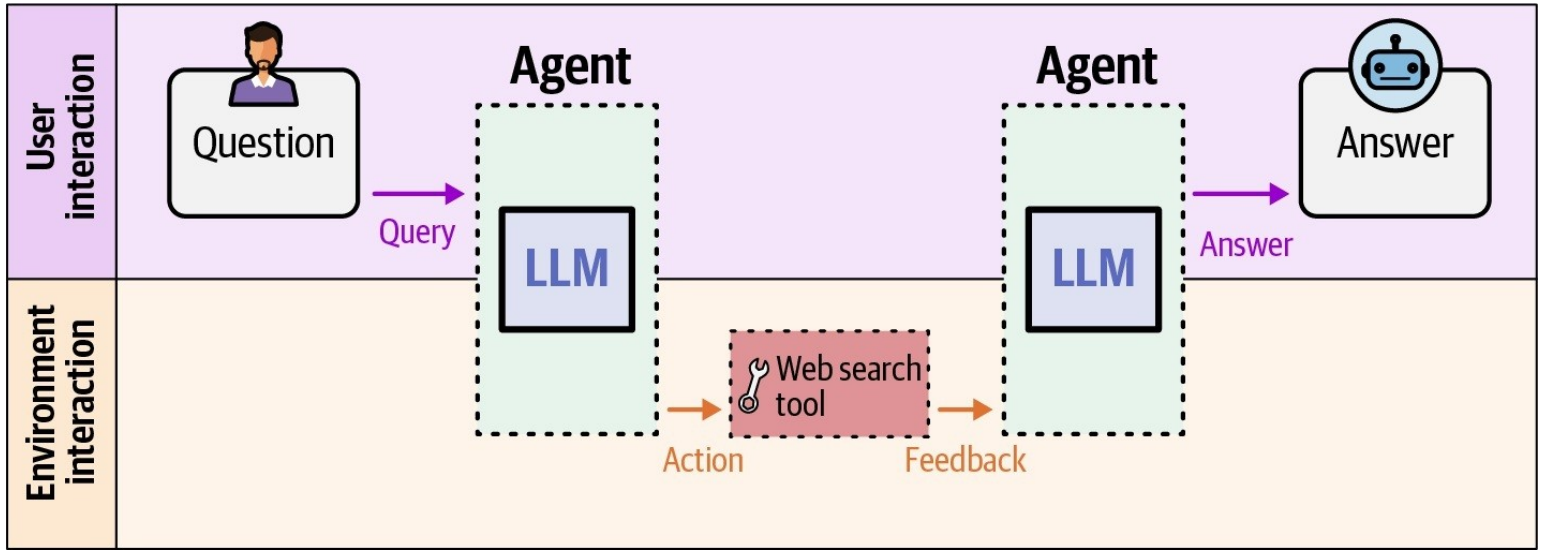
Fixed workflow
A developer specifies the sequence in advance. Each stage runs in a predefined order, even when an LLM performs work inside a stage.

Adaptive execution
An agent selects the next tool using the current goal and the results obtained so far. The sequence can change between tasks.

Choice of approach
A routine, well-specified process may fit a fixed workflow. Open-ended work benefits when the system can investigate and adapt its next action.

Grootendorst & Alammar (2026). Fig. 5-2, PDF p. 240 and Fig. 5-3, PDF p. 240

A research task becomes an interaction loop



Goal and information gap
A user asks a question. The model recognizes that answering requires information beyond the current context.

Action and observation
The model requests a search. Software runs the search and returns its result as an observation that the model can process.

Completion
The model uses that evidence to produce an answer or to decide on another action. Coding and automation use the same general pattern with different environments and tools.

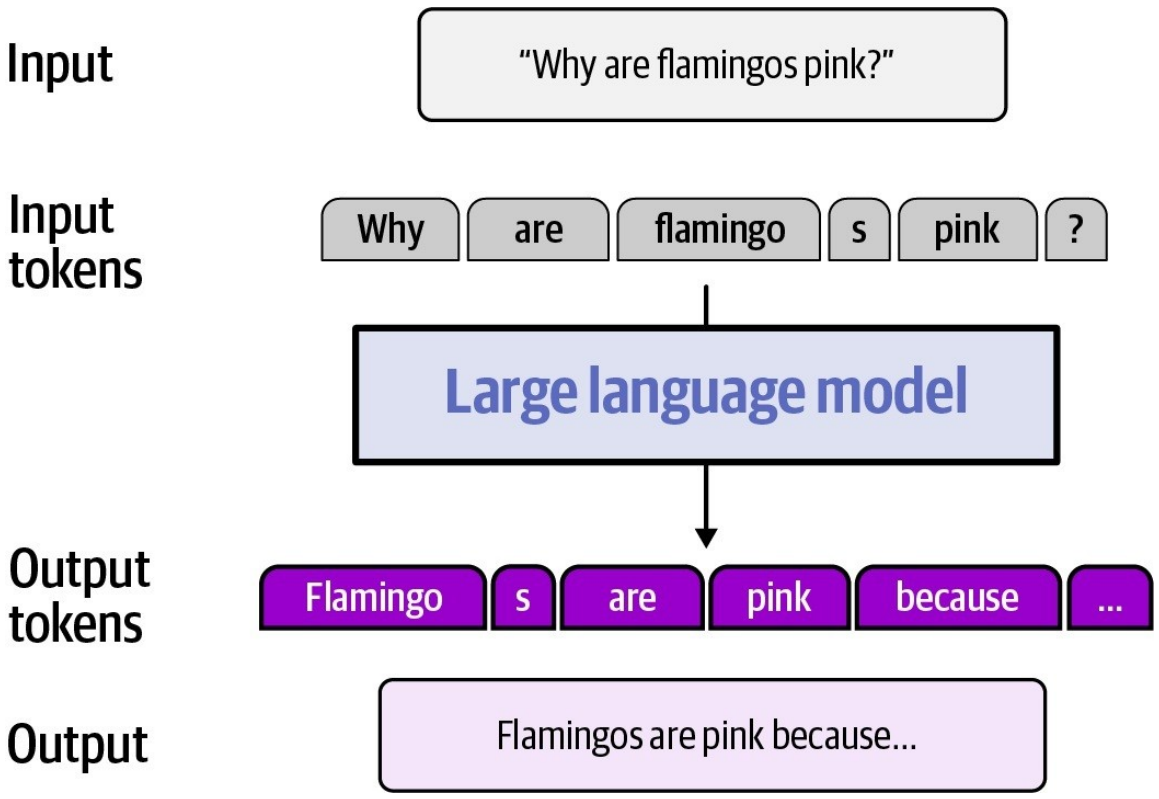
Grootendorst & Alammar (2026). Fig. 2-2, PDF p. 51

Tokens are the units of model input and output

Tokenization
A tokenizer splits text into units such as words, word fragments, punctuation, or numbers. A familiar word may require several tokens.

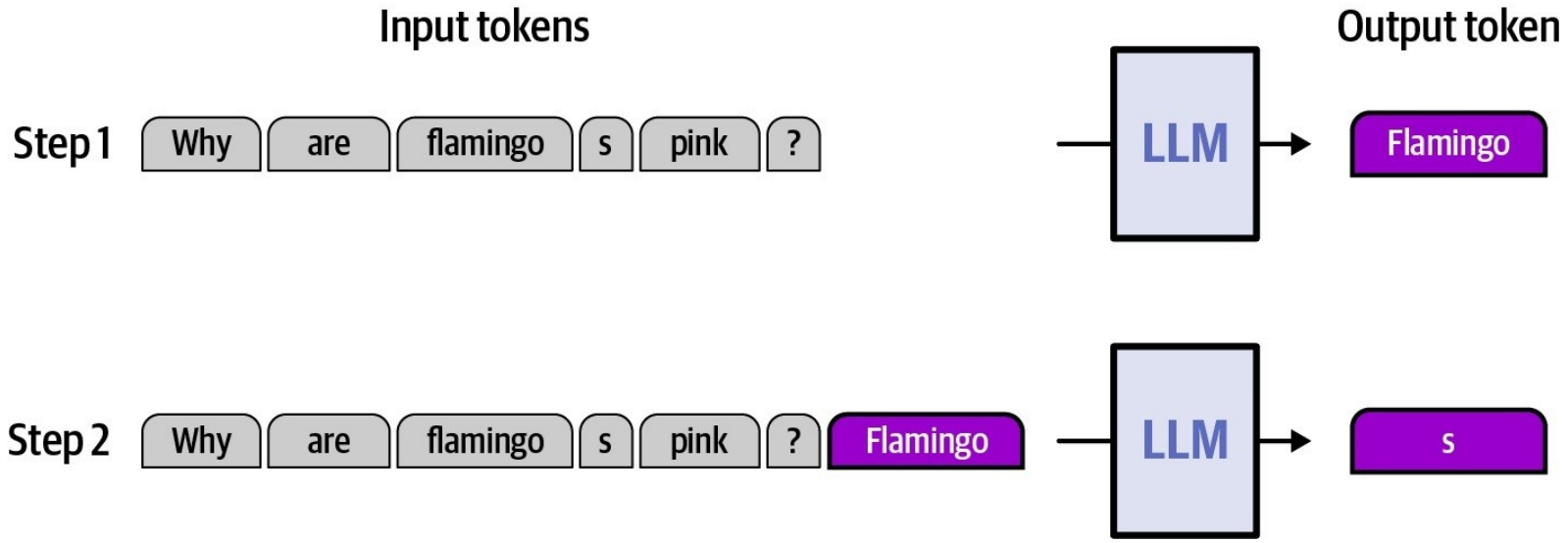
Representation
The model receives token IDs rather than raw strings. The tokenizer vocabulary determines which IDs represent the available pieces of text.

Operational consequence
Prompts, conversation history, tool definitions, and generated responses all consume tokens. Token counts help explain context limits and the amount of processing required.



Grootendorst & Alammar (2026). Fig. 2-3, PDF p. 53

Autoregression builds an answer one token at a time



One generation step
The model processes the available sequence and predicts a distribution for the next token. A decoding strategy selects a token.

Repeated generation
The selected token joins the sequence. The model continues generating from this extended context until it reaches a stopping condition.

Why the sequence matters
Earlier generated text influences later output. Intermediate reasoning and tool results can therefore shape what the model generates next.

Grootendorst & Alammar (2026). Fig. 2-4, PDF p. 54

Messages and system prompts structure interaction

Roles

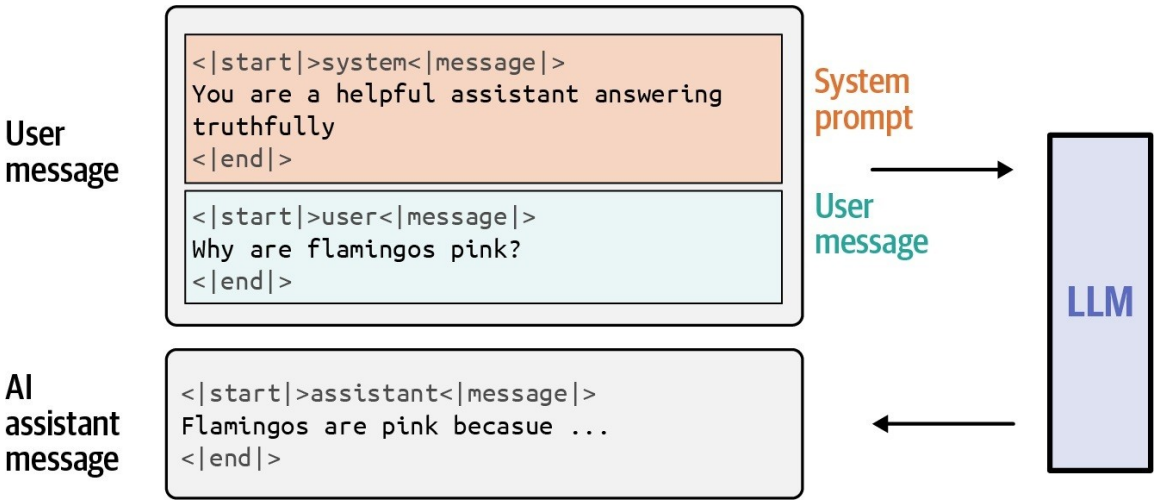
A conversation distinguishes messages from the system, the user, and the assistant. Tool-capable systems also represent tool requests and results.

System prompt

The deployer supplies instructions that shape expected behavior. This customizes model use without updating the underlying weights.

Chat template

A model-specific template converts structured messages into the token sequence the model expects. Role boundaries and special tokens help identify who is speaking and what output should follow.



Grootendorst & Alammar (2026). Fig. 2-5, PDF p. 55

A Transformer LLM has a processing pipeline

Tokenizer and embeddings

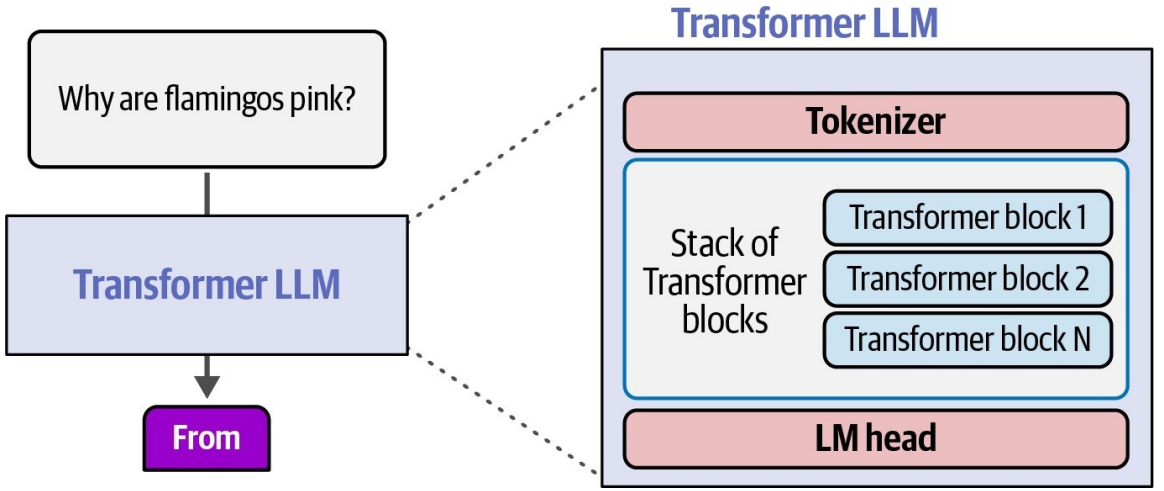
Text becomes token IDs, and each ID selects a numeric vector. These vectors enter the neural network.

Transformer blocks

A stack of blocks progressively transforms the representations using context from the sequence and learned model parameters.

Language modeling head

The final representation produces scores over the vocabulary. Token selection converts those scores into the next piece of generated text.



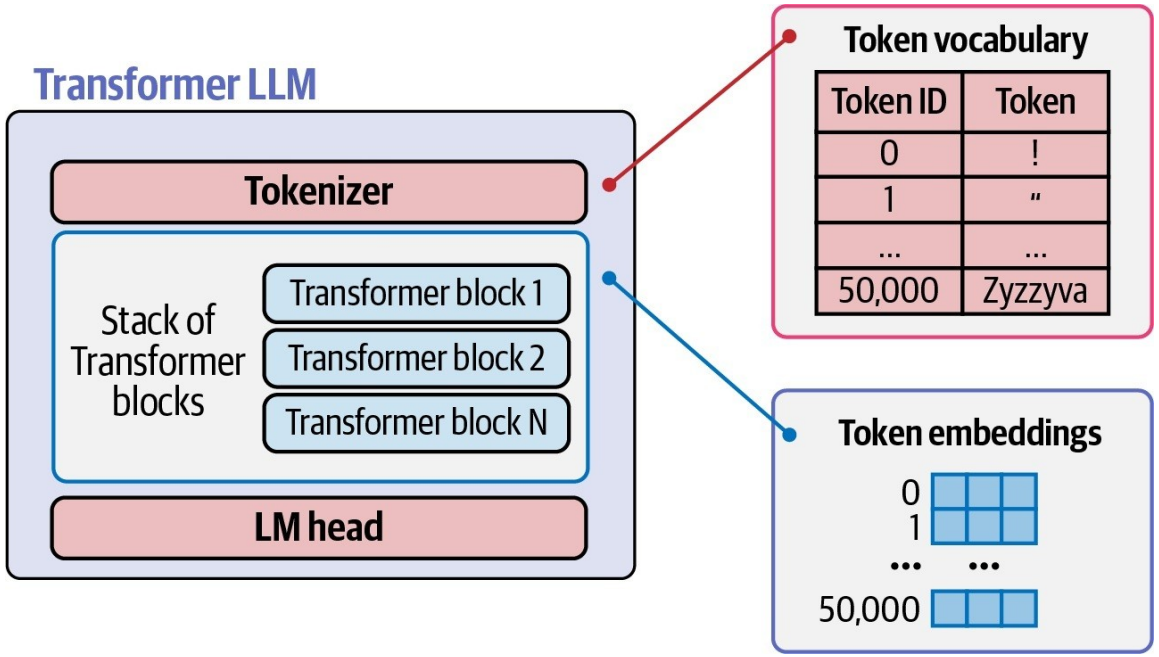
Grootendorst & Alammar (2026). Fig. 2-14, PDF p. 77

Embeddings make tokens numerically processable

Lookup
An embedding table associates each vocabulary token with a learned numeric vector. The token ID selects the corresponding row.

Initial representation
The same vocabulary token begins with its stored embedding. Transformer processing then changes its representation in light of the surrounding sequence.

A useful distinction
Token embeddings represent the units entering an LLM. Retrieval systems can use separate embedding models to represent whole passages for similarity search.



Grootendorst & Alammar (2026). Fig. 2-15, PDF p. 78

Transformer blocks refine representations

Self-attention

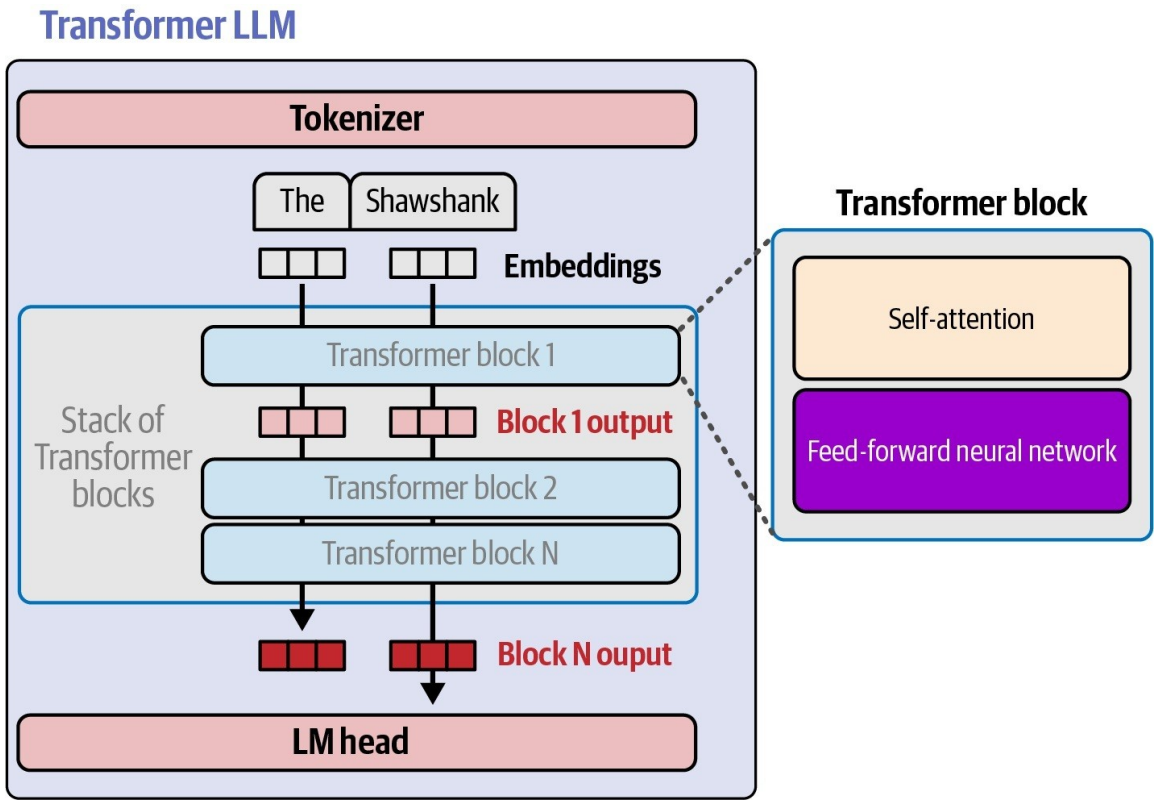
Each token can draw information from relevant positions in its context. This allows its representation to reflect relationships within the sequence.

Feed-forward network

The block also applies learned transformations to each token representation. Training shapes these transformations to capture patterns in the data.

Repeated depth

A stack of blocks repeats this process. Representations retain their interface dimensions as they pass from one block to the next.



Self-attention uses context to resolve meaning

Context dependence

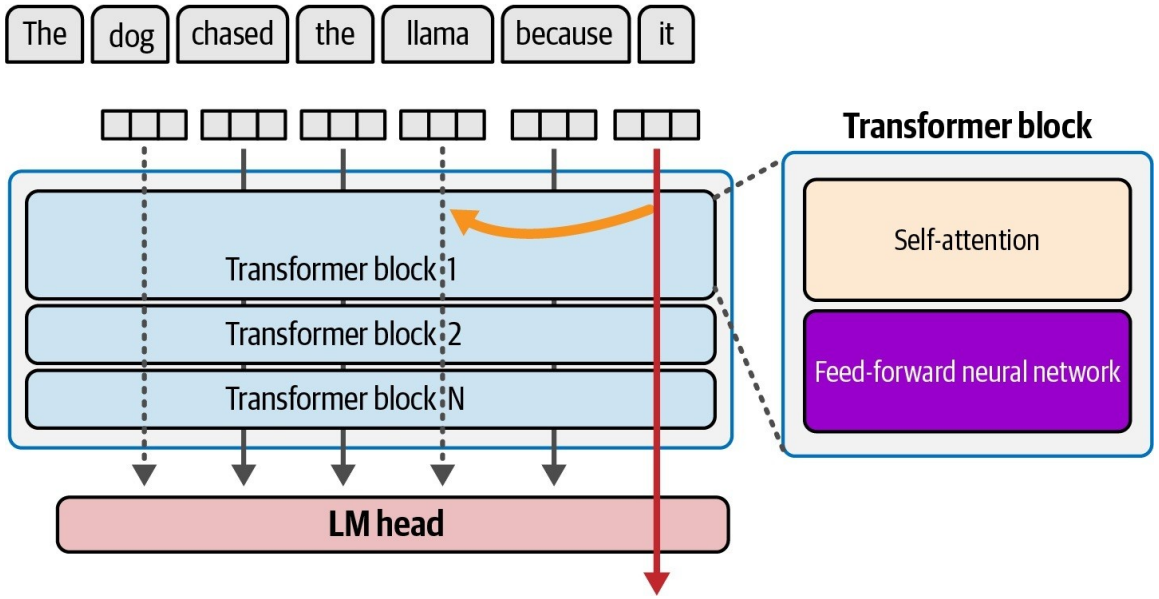
The meaning of a word or pronoun depends on surrounding tokens. A token processed in isolation lacks those relationships.

Attention weighting

Self-attention computes which earlier positions are relevant and combines information from them into the current representation.

Multiple perspectives

Different attention heads can learn different patterns of relevance. Their outputs help the model form a richer contextual representation.



Grootendorst & Alammar (2026). Fig. 2-26, PDF p. 93

Queries, keys, and values implement attention

Relevance scoring

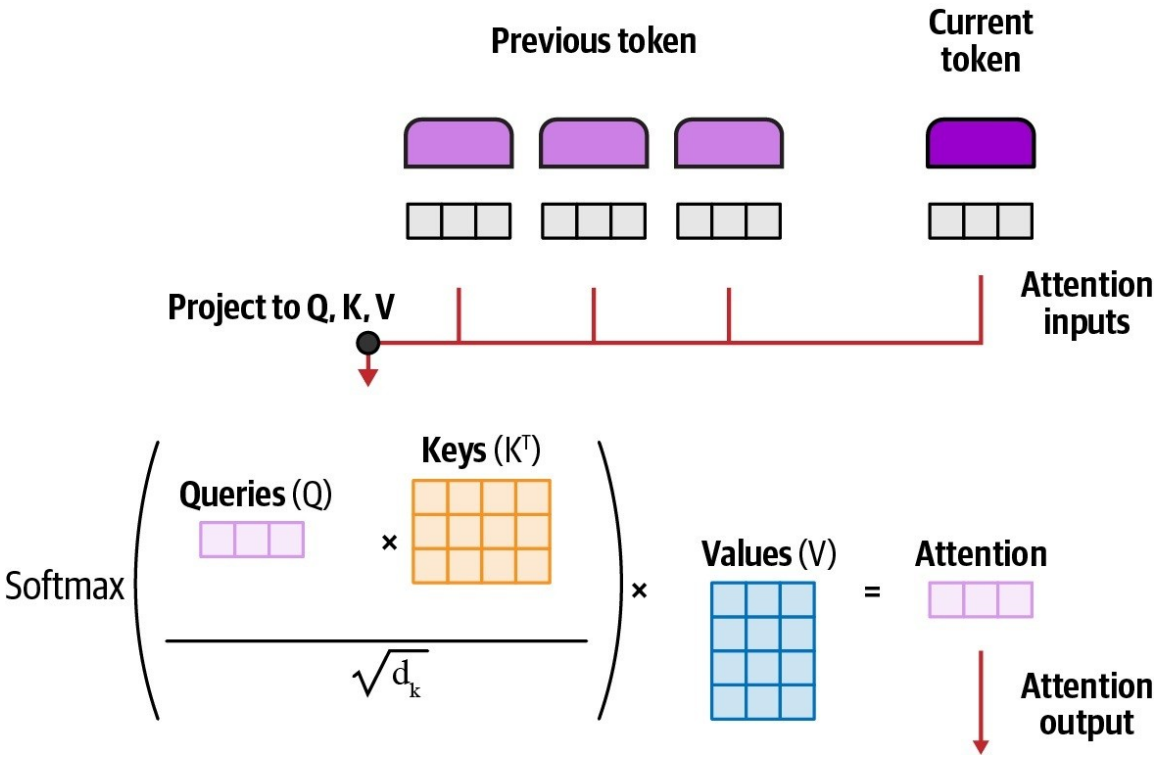
Learned projections create queries, keys, and values from token representations. The current query is compared with keys to score relevant context.

Information combining

Normalized relevance weights scale the value vectors. Combining these weighted values produces contextual information for the current token.

Causal generation

A decoder generating the next token uses available earlier positions. It cannot rely on tokens that have not yet been generated.



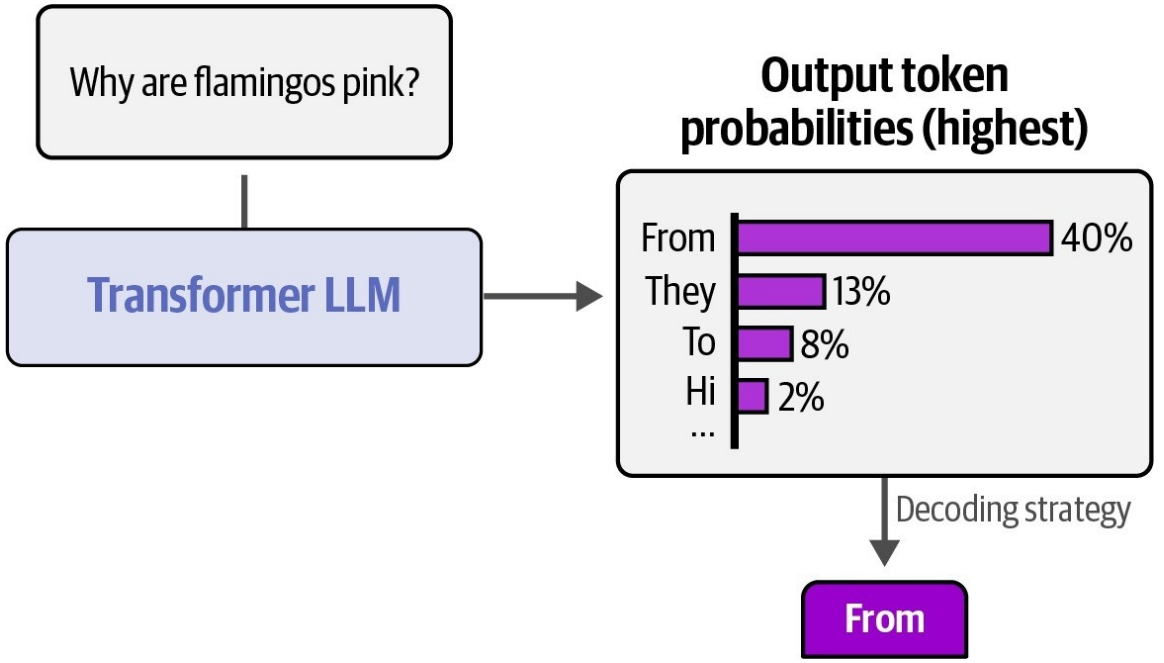
Grootendorst & Alammar (2026). Fig. 2-33, PDF p. 100

Decoding selects from the next-token distribution

Prediction
The language modeling head scores every token in the vocabulary. The highest score identifies the most likely continuation under the model.

Temperature
Low temperature concentrates selection on high-probability choices. Higher temperature increases diversity by giving lower-probability tokens more opportunity.

Use in reasoning
Diverse sampling helps generate alternative candidates for later verification. Diversity alone does not establish correctness or improve every task.



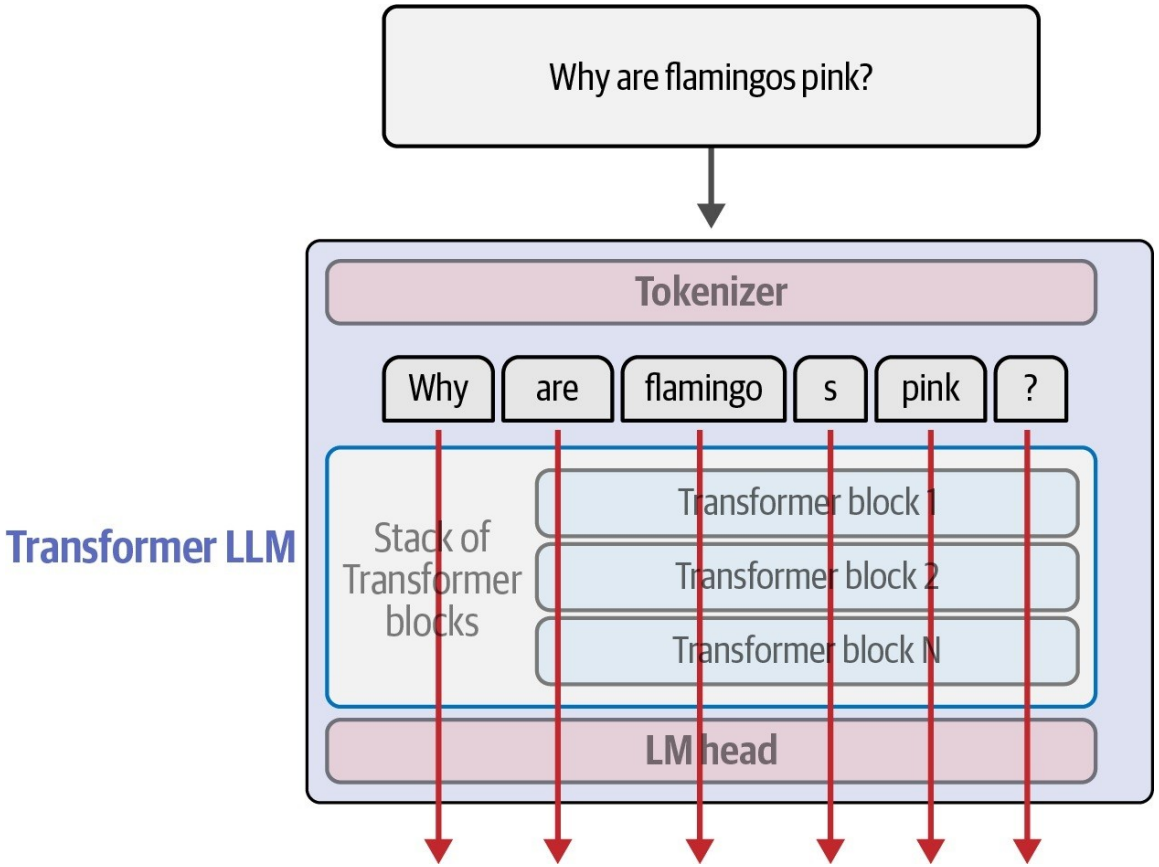
Grootendorst & Alammar (2026). Fig. 2-17, PDF p. 80

Context length limits what a call can use

Available information
The context contains the current request and any supplied history, instructions, evidence, and intermediate outputs.

Finite capacity
A model supports a maximum sequence length. Input and generated output need to fit within the relevant model limits.

Capacity and usefulness
A large context window allows more information, but it also increases processing demands. Selecting relevant context remains necessary as tasks and trajectories grow.



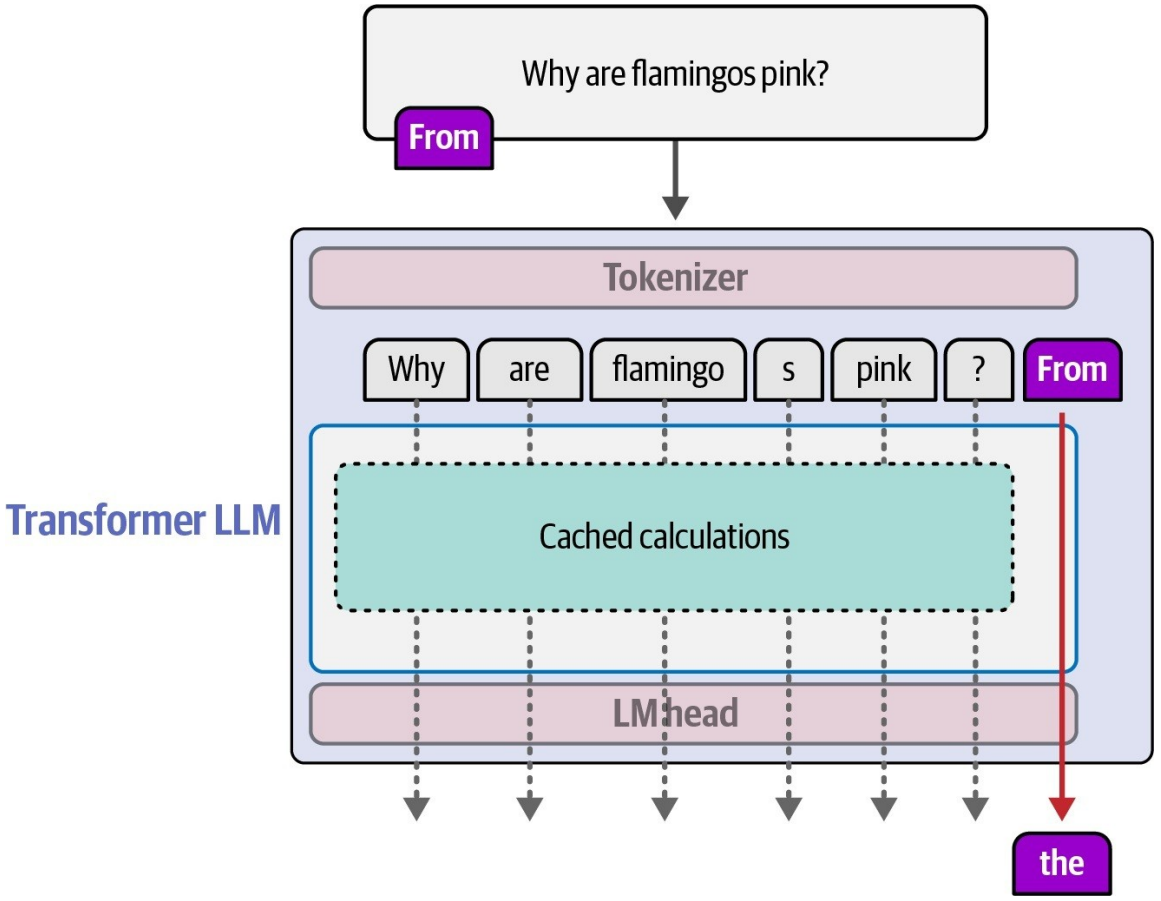
Grootendorst & Alammar (2026). Fig. 2-18, PDF p. 81

The KV cache avoids repeated computation

Reusable work
Earlier token positions already have computed keys and values. A cache retains them for later generation steps.

New-token processing
The model processes the new token and uses the cached context. This reduces repeated work compared with recomputing earlier keys and values.

Agent consequence
Long agent trajectories repeatedly reuse context. Cache-aware context organization can reduce latency and cost, while the cache itself consumes memory.



Grootendorst & Alammar (2026). Fig. 2-20, PDF p. 85

Efficient attention changes the memory trade-off

Sharing keys and values

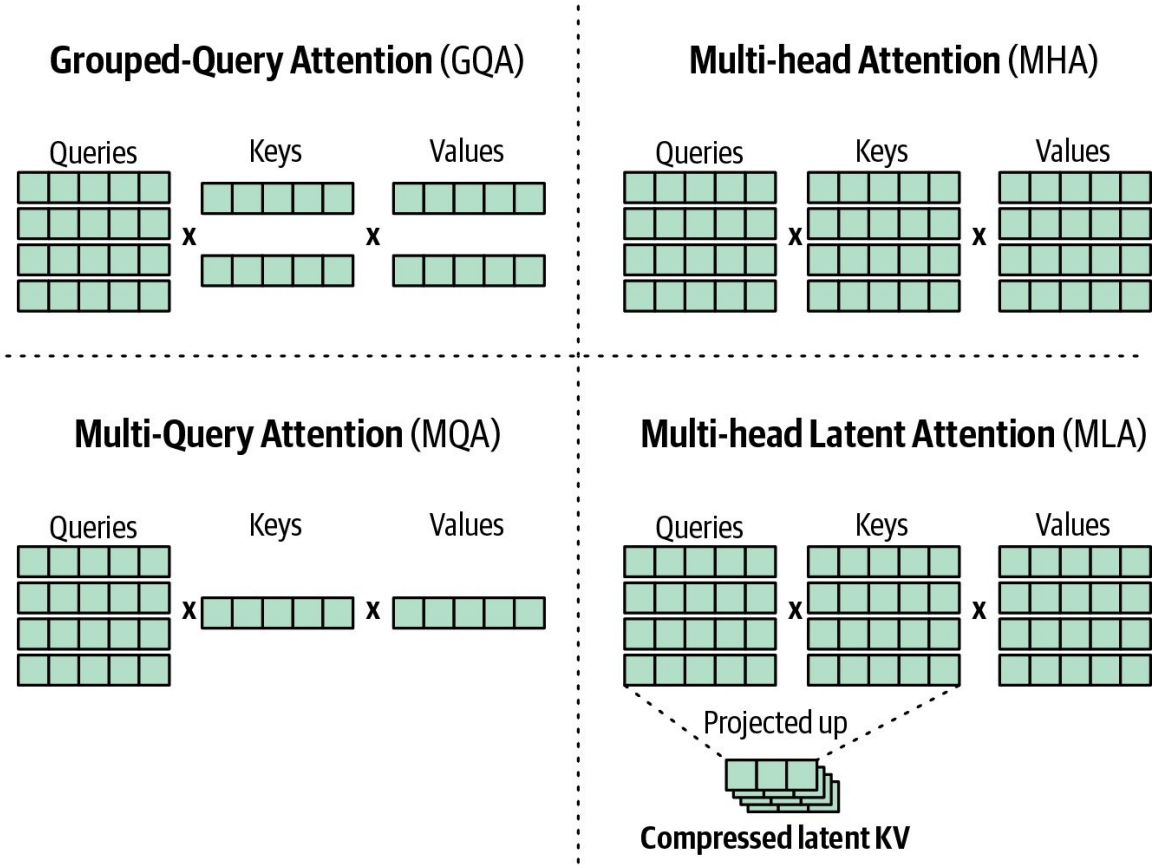
Grouped-query and multi-query attention share K and V across groups of query heads or across all query heads to reduce cache requirements.

Compressed representations

Multi-head Latent Attention stores a smaller latent representation of keys and values rather than their full forms.

Selective attention

Sparse attention can select a subset of earlier tokens for expensive attention calculations. These design choices address different parts of the compute and memory problem.



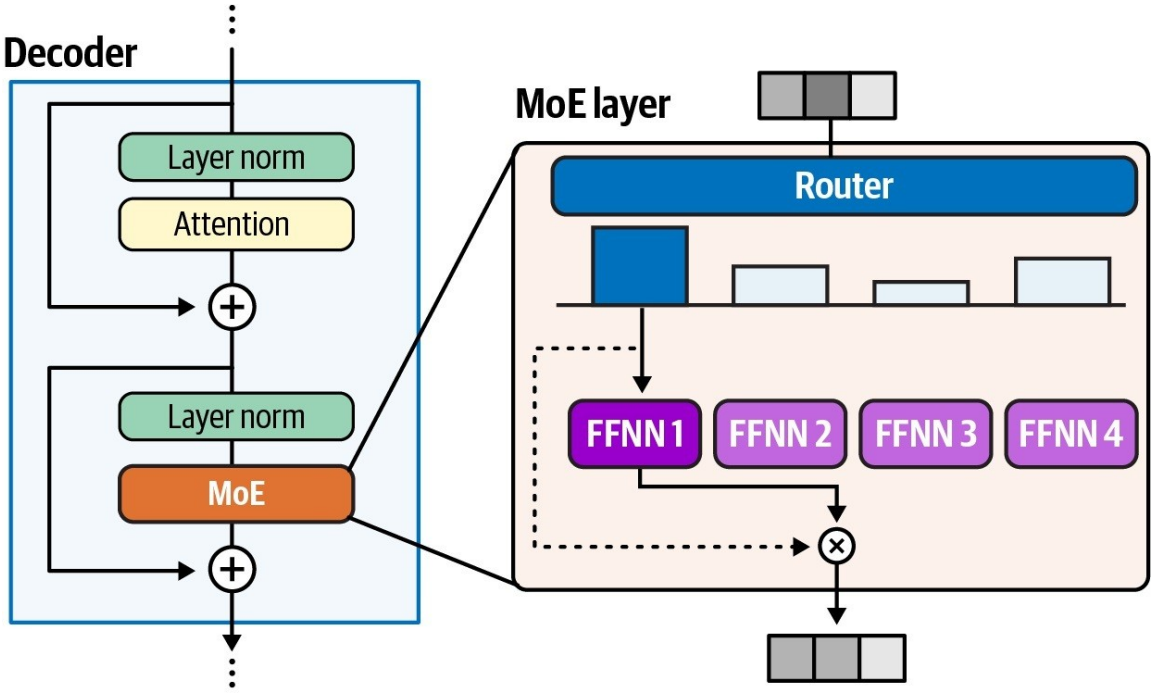
Grootendorst & Alammar (2026). Fig. 2-37, PDF p. 105

Mixture of experts activates a subset of networks

Experts
An MoE layer contains multiple feed-forward networks. A learned router scores which experts should process each token.

Sparse activation
Only a selected subset operates for a given token. Total parameter count can therefore be much larger than the active parameter count.

Deployment implications
Sparse activation can reduce computation, while storing the model still requires substantial memory. Expert load balancing helps prevent a few experts from receiving nearly all training traffic.



Grootendorst & Alammar (2026). Fig. 2-41, PDF p. 111

Pre-training learns next-token prediction

Training examples

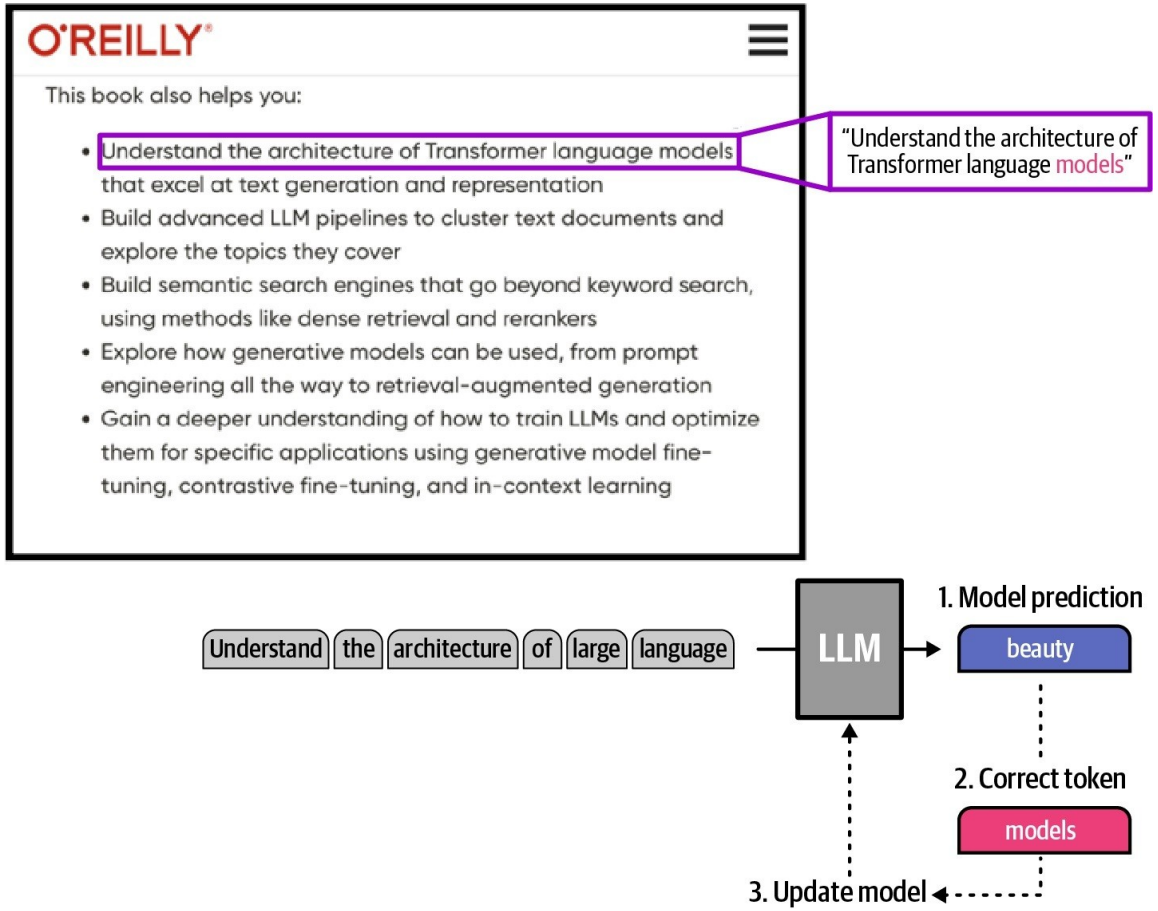
A sequence provides context, and the next token supplies the target. Large curated mixtures can include natural language and code.

Weight updates

The model compares its prediction with the target and updates its weights to reduce prediction error across many examples.

Base model

This stage produces broad language capabilities. Further training shapes how the model follows instructions and behaves in interactive applications.



Grootendorst & Alammar (2026). Fig. 2-8, PDF pp. 67–68

Supervised fine-tuning demonstrates desired behavior

Prompt and completion

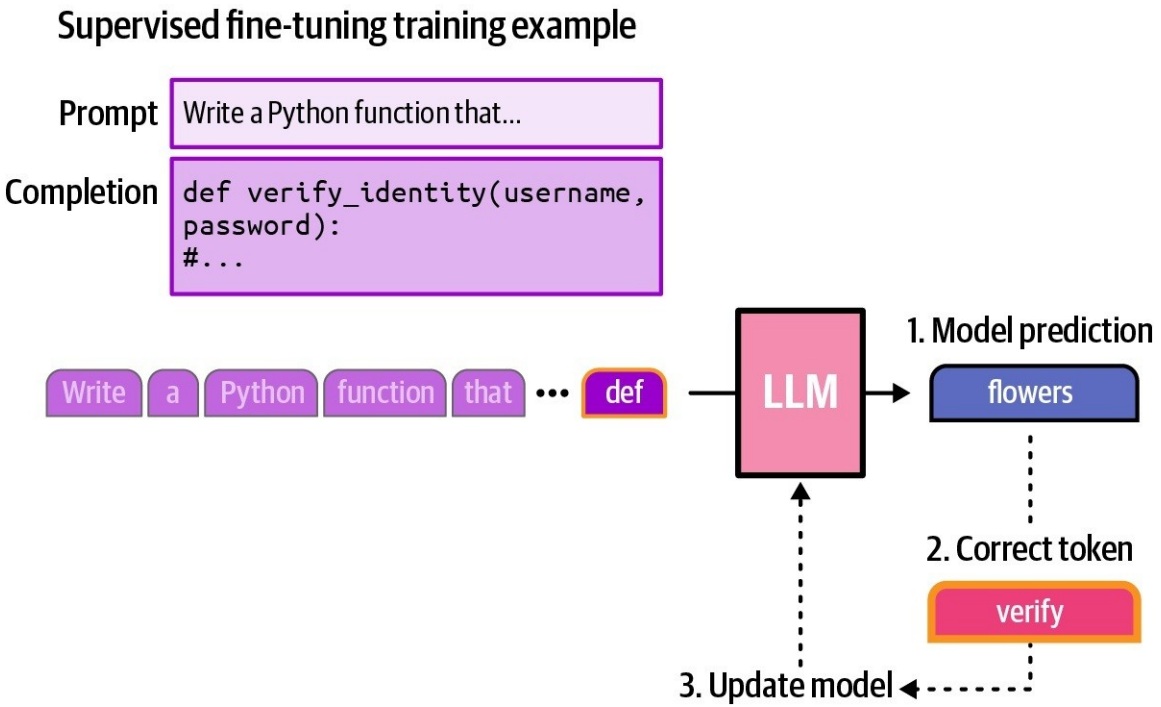
Supervised fine-tuning (SFT) pairs a request with a desired response. Examples can demonstrate instructions, dialogue, or reasoning.

Learning signal

The prompt supplies context. The training loss focuses on the target completion so the model learns the demonstrated response behavior.

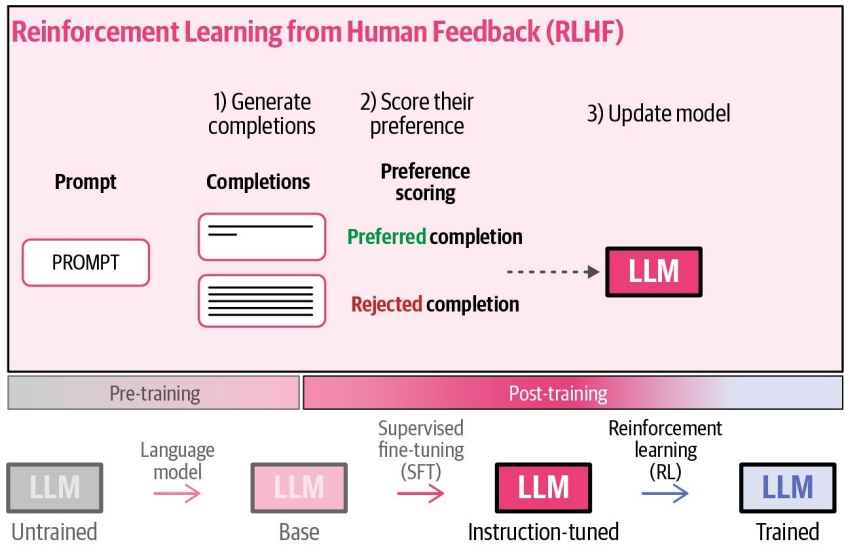
Scope and limits

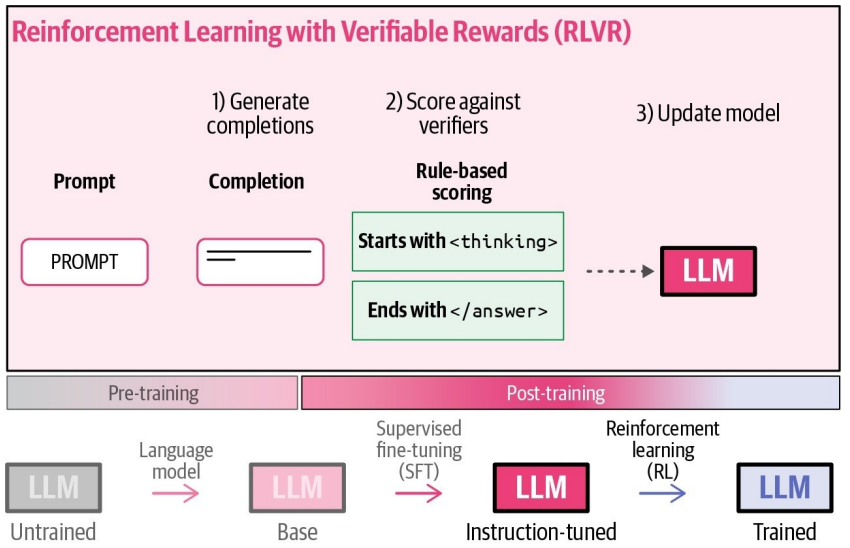
SFT makes desirable patterns easier to elicit. The quality and coverage of the examples influence how well that behavior generalizes.



Grootendorst & Alammar (2026). Fig. 2-9, PDF p. 69

Reinforcement learning uses feedback on outputs





Human preference

RLHF uses preference information, often through a reward model, to favor more useful or appropriate responses. It does not require a single complete target response.

Verifiable reward

RLVR scores outputs with automated checks such as answer correctness, required formatting, or passing code tests.

Different signals

Preference and verifiable correctness measure different properties. A useful reward must represent the behavior the training process is intended to improve.

Grootendorst & Alammar (2026). Fig. 2-10, PDF p. 70 and Fig. 2-11, PDF p. 72

GRPO compares a group of candidate responses

Sampling

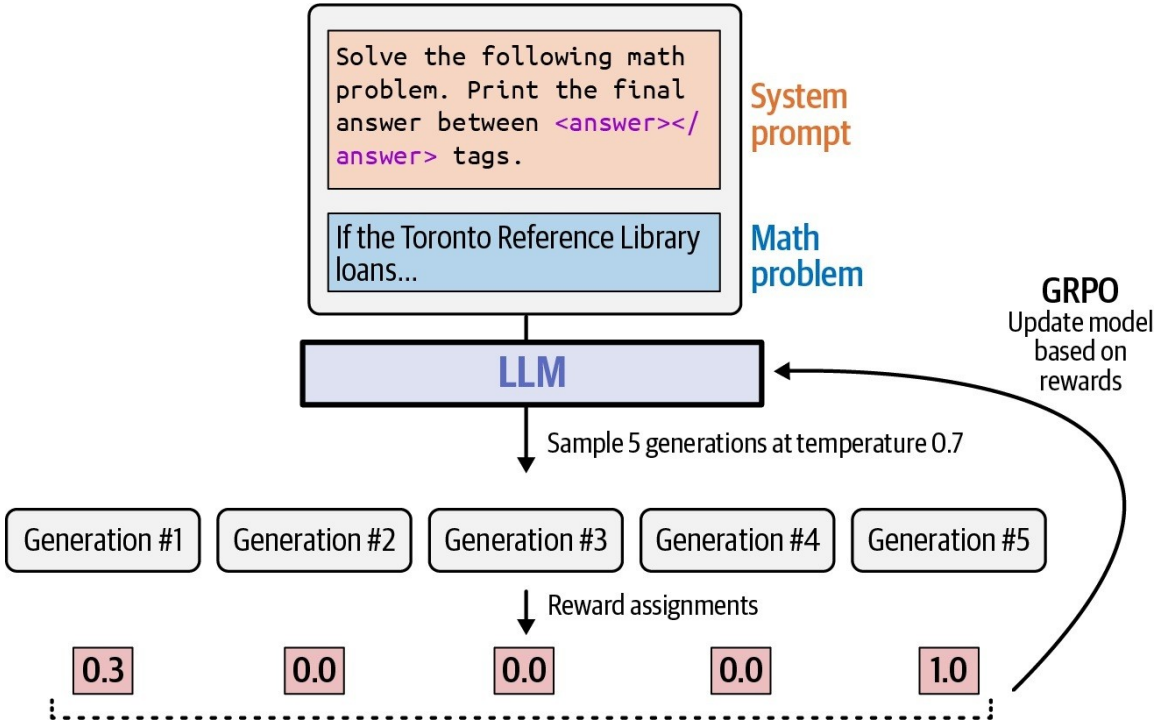
The model generates several responses for the same prompt. Variation gives the learning process different behaviors to compare.

Relative feedback

A reward function scores the responses. Group Relative Policy Optimization reinforces responses that perform better relative to others in that group.

Reasoning application

Format and accuracy rewards can encourage useful reasoning behavior without providing every intermediate reasoning step as a supervised target.



Grootendorst & Alammar (2026). Fig. 2-13, PDF p. 76

Reasoning models generate intermediate computation

Deliberate generation

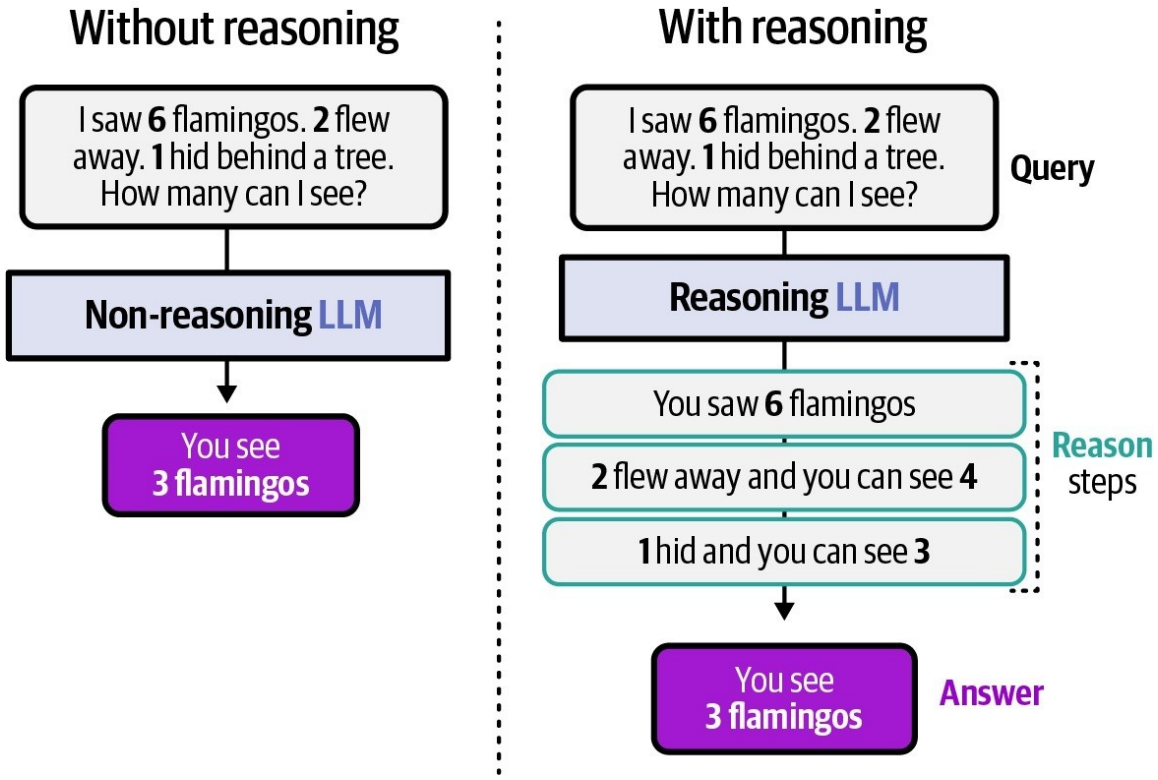
A reasoning model produces intermediate tokens before its final answer. These tokens can help represent subproblems and relationships.

Agent relevance

Planning, tool selection, and reflection all require decisions that can benefit from extended computation.

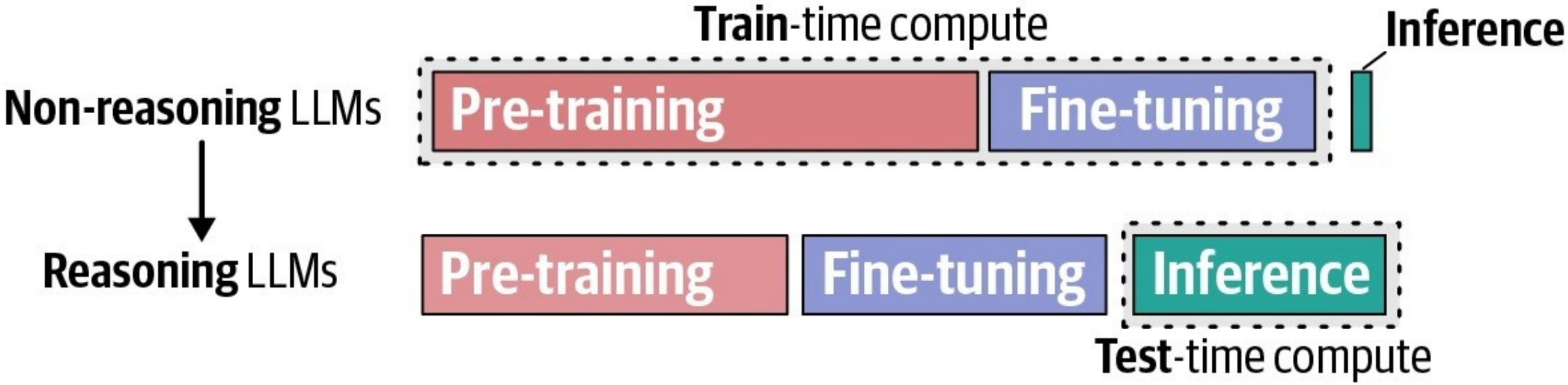
Cost and interpretation

Reasoning takes time and tokens. A model can still confidently generate an incorrect claim, a hallucination. Generated intermediate reasoning can also contain errors.



Grootendorst & Alammar (2026). Fig. 3-1, PDF p. 123

Training and inference use compute differently



Train-time compute
More training data, model capacity, or optimization changes the model before deployment. Post-training also contributes to this budget.

Test-time compute
More inference work can mean a longer reasoning process, more candidate responses, or a more extensive search for a solution.

The trade-off
Extra inference work can improve difficult tasks when it supports useful computation. It also increases latency and resource use, and its benefit depends on the task and method.

Grootendorst & Alammar (2026). Fig. 3-10, PDF p. 130

Prompting can elicit structured reasoning

Examples as guidance

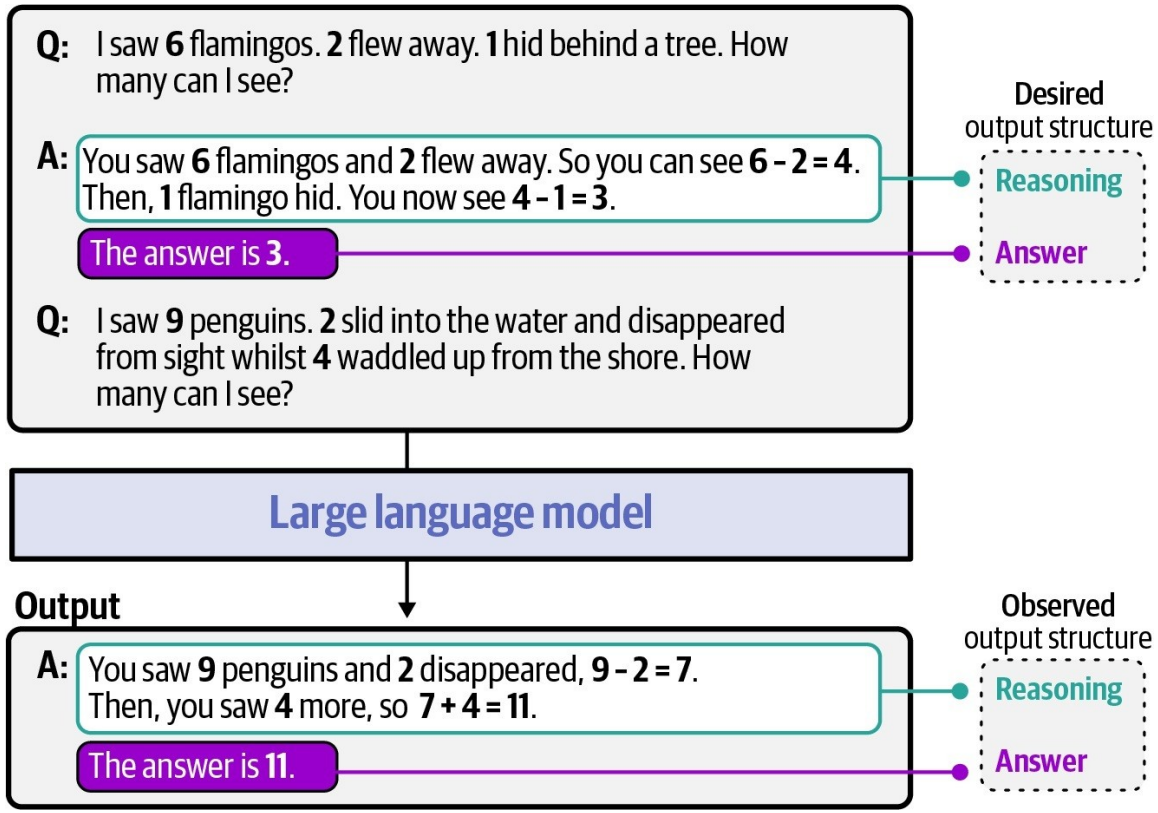
Few-shot prompts demonstrate how to approach a task. The model can imitate the structure of the examples without a weight update.

Stepwise decomposition

Chain-of-Thought prompting encourages intermediate steps. Zero-shot variants ask for a stepwise approach without providing worked examples.

Practical boundary

Prompting depends on instruction following and task fit. A successful demonstration does not establish that the same prompt will work reliably across all questions.



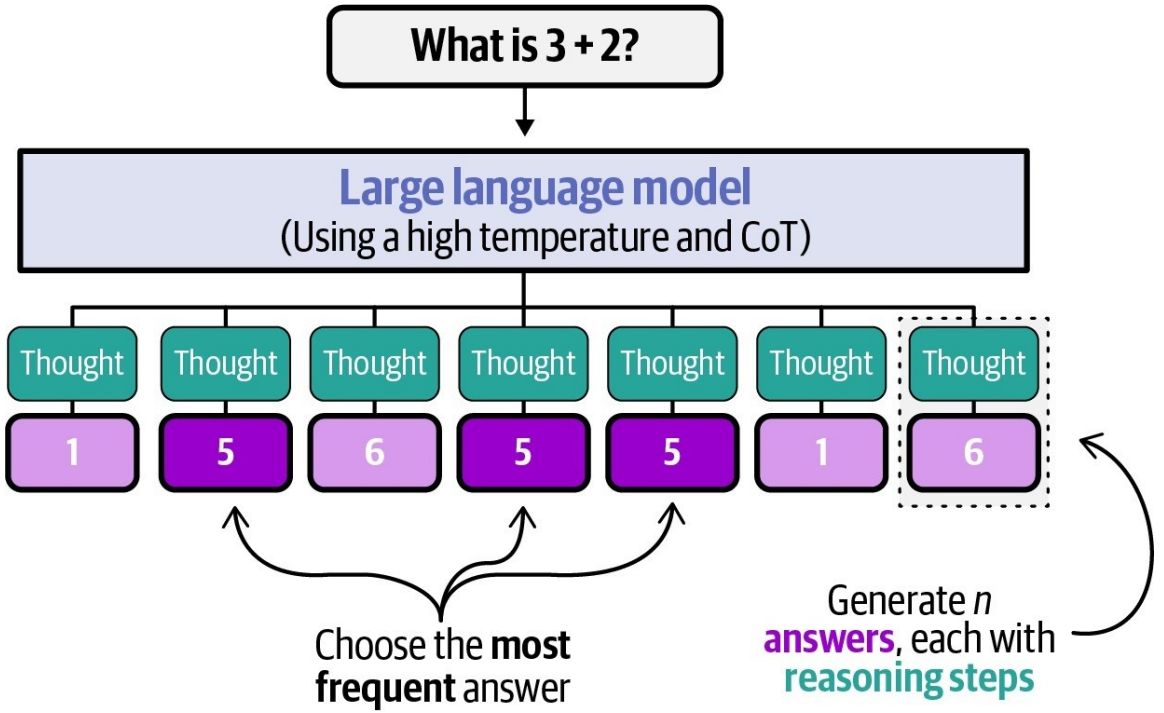
Grootendorst & Alammar (2026). Fig. 3-22, PDF p. 144

Self-consistency selects the most frequent answer

Candidate generation
The system samples multiple reasoning paths and extracts the resulting answers. Sufficient variation matters if the samples are to explore alternatives.

Selection
It chooses the answer that appears most often. This method can reduce sensitivity to a single unsuccessful generation.

Limitation
Agreement is a selection signal. Multiple candidates can share the same mistake, so the most frequent answer still needs appropriate validation.



Grootendorst & Alammar (2026). Fig. 3-25, PDF p. 151

Verifiers compare outcomes or intermediate processes

Outcome reward model

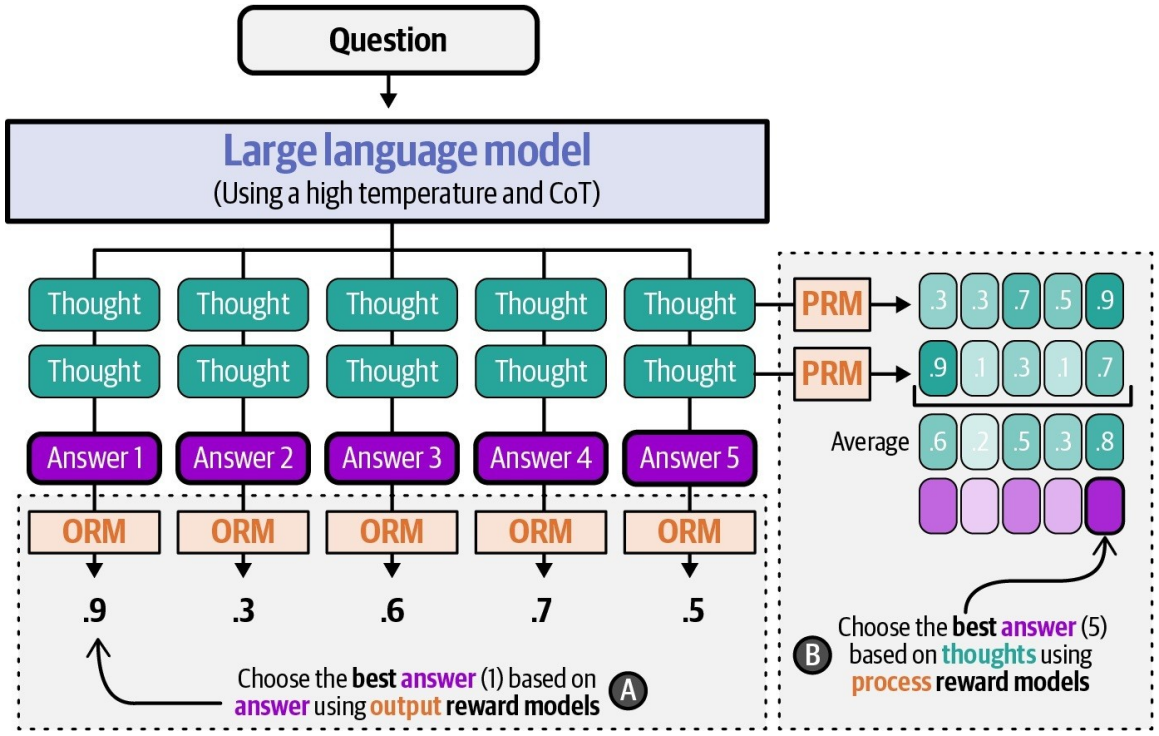
An ORM scores the final result. Tests for a generated function are an example of checking whether an output behaves correctly.

Process reward model

A PRM assesses intermediate reasoning steps. It can provide a finer signal about where a candidate becomes promising or unsound.

Best-of-N

The system generates N candidates, scores them, and selects the highest-scoring one. Its quality depends on both candidate diversity and verifier quality.



Training can make reasoning a native behavior

Demonstration and exploration

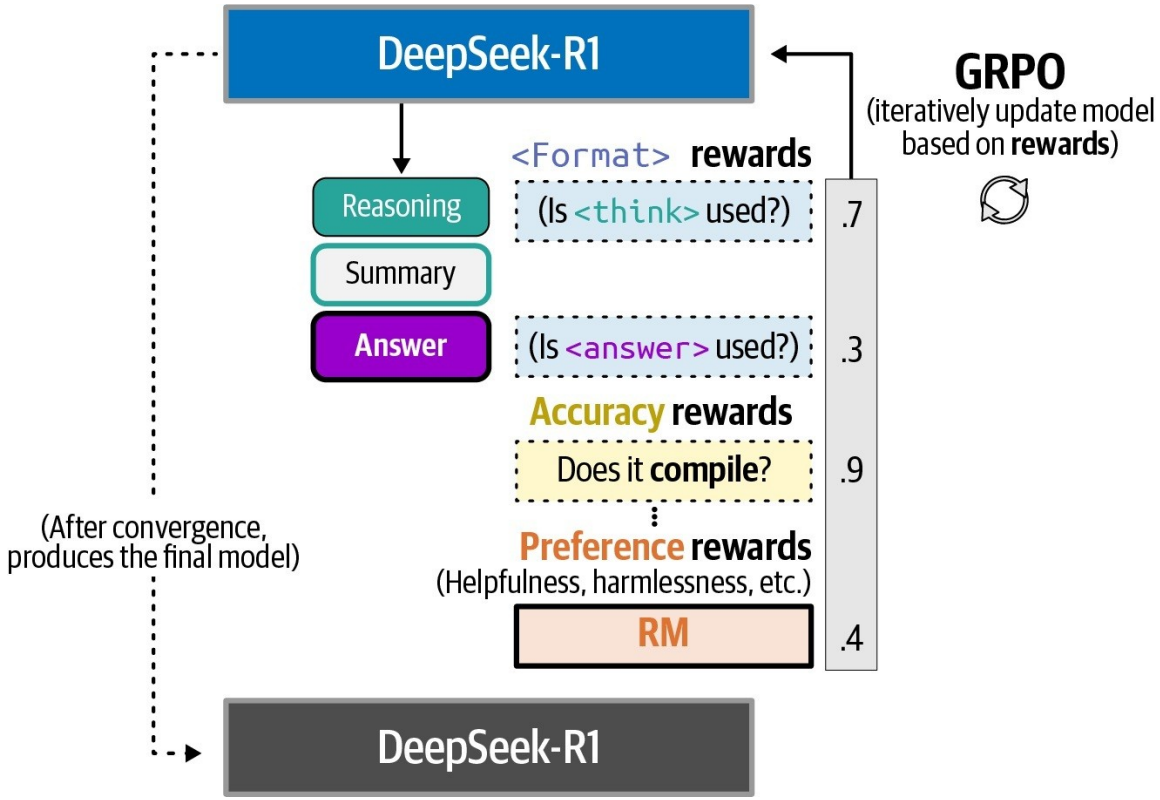
SFT teaches useful reasoning patterns. RL can reward successful solutions and encourage strategies beyond direct imitation.

DeepSeek-R1 case

A cold-start SFT stage improves readability, reasoning-focused RL strengthens problem solving, and generated reasoning plus general-task data support further SFT and final RL.

Operational interface

A trained model uses its expected chat template and reasoning controls. Application software still supplies context and handles the model output.



Grootendorst & Alammar (2026). Fig. 3-41, PDF p. 172

Efficient reasoning balances length and usefulness

Overhead

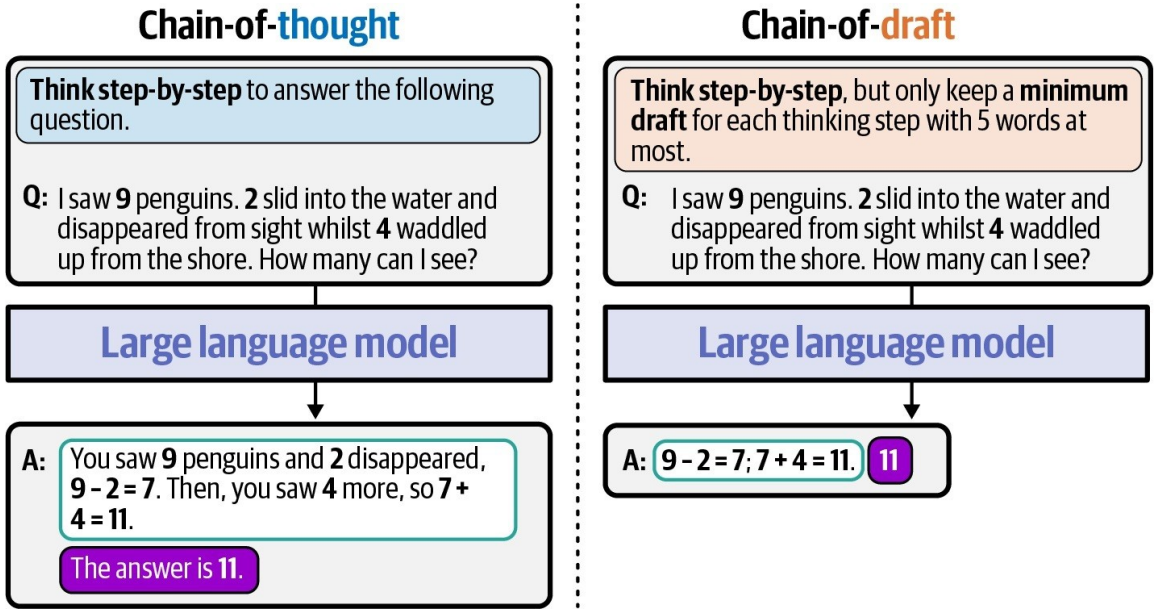
Long reasoning traces can contain repetition and consume resources without adding useful progress.

Concise reasoning

Chain-of-Draft uses short intermediate drafts. Token budgets, length rewards, and hybrid thinking modes provide other ways to control inference work.

Quality constraint

Reducing length can discard necessary reasoning on hard problems. Efficiency should be judged together with task success, rather than from token count alone.



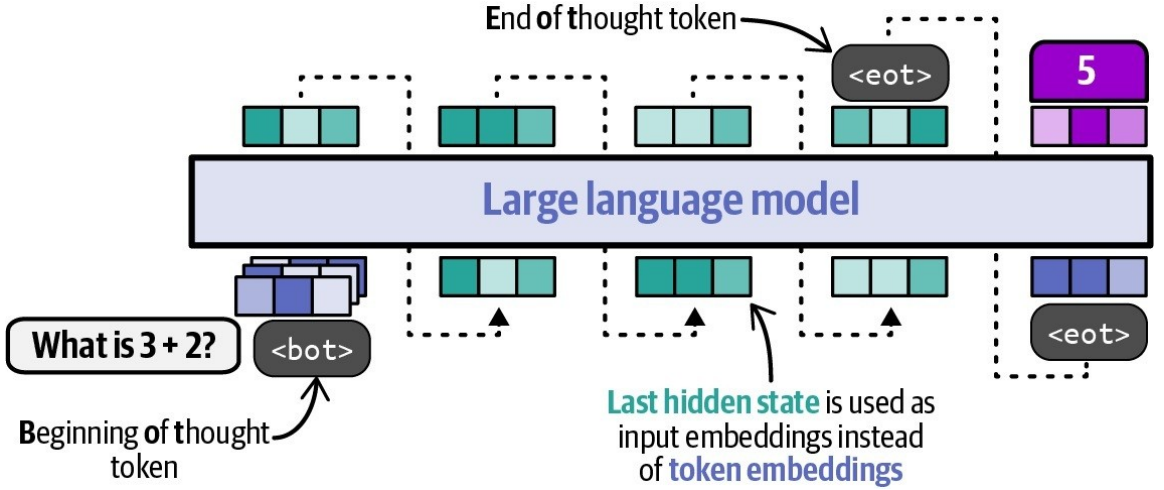
Grootendorst & Alammar (2026). Fig. 3-46, PDF p. 180

Reasoning research extends beyond visible text

Multimodal reasoning
Models can reason using representations of images alongside text. Training data and reward signals must suit the kinds of input and task.

Latent reasoning
Continuous-thought approaches pass hidden representations forward during reasoning instead of decoding every intermediate step into visible tokens.

Trade-off
Compact internal computation may improve efficiency, while visible intermediate output can help inspection. These are research directions with distinct design choices and limitations.



Grootendorst & Alammar (2026). Fig. 3-49, PDF p. 183

Learning check with worked answers

1. Why does a chat model appear to remember?
The application can resend relevant history as context. The next call uses that supplied information.

2. Does a generated function call perform an action?
Execution requires external software that parses the request, calls the function, and returns a result.

3. Are MoE experts separate agents?
They are internal neural networks selected by a router. Multi-agent systems contain separately operating agents.

4. Why can longer reasoning still fail?
Extra tokens can repeat an error or add irrelevant work. Verification and task-appropriate computation remain necessary.

Core distinctions and source guide

Conceptual distinctions

The LLM generates decisions. A harness executes actions. Context supplies usable information. Training updates weights. Caching reuses computation. Autonomy concerns who chooses the next step.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapters 1–3. Selected connections: Chapters 4–5 and 10.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF